

DoctorlyCalendar — Review Day Cheat Sheet

- 45-minute review. They ask about your solution first, then general .NET.
 - Their rubric (from the email): "no right or wrong — we want to see your approach, your choices, and how you communicate."
 - Winning move: narrate your choices, point out your own gaps before they do, and end answers on a trade-off.
-

1. "Walk me through your solution" — recite point by point

- "It's a backend API for managing calendar events in a doctor's practice."
- "It's a single .NET 8 project, split into three folders so the layers are clear: **Domain, Infrastructure, Features**."
- "**Domain** holds the business — the `CalendarEvent` and `Attendee` entities, the enums, and the `INotificationService` interface. It has no infrastructure dependencies."
- "The domain model is **rich** — entities have behaviour, not just data. You create a `CalendarEvent` through a factory method, `Create()`, where validation runs. Setters are private, so state only changes through methods like `Update`, `Cancel`, and `AddAttendee`, each with its own guard rules."
- "`CalendarEvent` is the **aggregate root**. `Attendee` only exists inside it — you add one through `CalendarEvent.AddAttendee()`, which enforces the rules: no duplicate emails, and you can't add to a cancelled event."
- "**Infrastructure** holds the EF Core `DbContext`, the entity configurations, the repository, and the notification stub. `ICalendarEventRepository` hides EF Core behind an interface."
- "**Features** uses **vertical slices** — each operation has its own folder and controller: create, update, cancel, add-attendee, get, search. Controllers are thin and call the repository directly. I skipped MediatR because at this scale it adds ceremony with no value."
- "For persistence I used **SQLite** so it runs with no setup — the database creates itself on startup via `db.Database.Migrate()`. In production I'd switch the provider to SQL Server or Postgres; only the connection string and package change."
- "For concurrency I used **optimistic concurrency** with a `RowVersion`. SQLite has no native rowversion, so it's a Guid I regenerate on every write, marked as a concurrency token. On update the client sends back the `RowVersion` it has; if it doesn't match, I return **409**."
- "**Notifications** go through `INotificationService`, defined in Domain and implemented in Infrastructure as a logging stub — so swapping to email or a queue is a one-line change in `Program.cs`."
- "I **tested the domain logic** directly with xUnit and Shouldly — no mocks needed, because the domain has no dependencies."
- "I wrote up my **decisions and known next steps in the README** — moving to a native rowversion, auth, pagination, and adding `DomainExceptions`."

Keep it to about 60–90 seconds. Then pause and let them ask. Don't over-talk.

2. What your solution does well — lead with these

- **Single project, clean folders:** Domain, Infrastructure, Features. Fast, still clearly separated.
 - **Rich domain model (real DDD):** private constructors, factory methods, private setters, guard clauses.
 - **Aggregate done right:** CalendarEvent is the root; Attendee is created and accessed only through it.
 - **Vertical slice:** one folder + controller per operation. Easy to find and change one feature.
 - **Optimistic concurrency:** Guid RowVersion marked IsConcurrencyToken().
 - **SQLite + migrate-on-startup:** reviewer clones and runs, no DB setup.
 - **Dependency Inversion:** INotificationService in Domain, implemented in Infrastructure.
 - **Domain tests:** pure C#, no mocks needed.
-

3. What your README already documents — claim these, don't apologise

You wrote these up as deliberate decisions and next steps. Present them as judgement, not things you forgot.

- SQLite limits (no native DateTimeOffset ORDER BY, no rowversion) + plan to move to SQL Server/Postgres.
 - Guid RowVersion instead of a native token, with the planned fix.
 - ICalendarEventRepository in Infrastructure is a pragmatic call (ideally in Domain) — documented honestly.
 - No auth, no pagination, soft delete, separate class libraries, real notifications — all listed under Improvements.
 - **"Add DomainExceptions" is already in your README** — so you are NOT claiming it exists. Framework exceptions now, DomainException is a planned next step. Make your spoken answer match this.
 - Adding Serilog and more domain tests — listed.
-

4. Real gaps to raise yourself — NOT in your README

Four genuine ones. Naming them first reads as senior.

Gap 1 — RowVersion is never returned to the client (the real bug) -

UpdateEventRequest and AttendeeRequest need a RowVersion, but CalendarEventResponse doesn't include it. - So a client can't get the current RowVersion to send back — the

concurrency round-trip is incomplete. - **Say:** "The token isn't surfaced on the response DTO, so the round-trip is incomplete. I'd add RowVersion to the response." (*Best to actually fix — Section 5.*)

Gap 2 — The concurrency exception isn't caught - The manual check handles a stale client (→ 409). But a true race — two clients save off the same version — is caught by EF at SaveChanges, which throws DbUpdateConcurrencyException. That's uncaught → **500, not 409**. - **Say:** "The manual check covers the common case; the EF token is the backstop for a real race, and I'd catch DbUpdateConcurrencyException and return 409."

Gap 3 — Domain rule errors return 500, not 400/409 - Bad input is caught at binding (→ 400). But rules inside the domain — whitespace title, end-before-start, duplicate attendee — throw exceptions that are uncaught → **500**. - Your README lists "Add DomainExceptions" but not the mapping. The fix is an exception-handling middleware that maps them to 400/409. - **Say:** "Domain-rule failures currently return 500. I'd add exception-handling middleware to map domain exceptions to the right status codes."

Gap 4 — Notifications only fire on Create - INotificationService has Updated/Cancelled/AttendeeAdded, but only CreateEventController calls the service. - **Say:** "I wired notifications on create to show the pattern; the rest is an identical quick add I left for time."

5. Fix before the review (then say you did)

- **Add RowVersion to CalendarEventResponse** and set it in the mappings (fixes Gap 1 — a real bug).
 - **Catch DbUpdateConcurrencyException** in update/add-attendee and return 409 (fixes Gap 2).
 - **Keep your exception story consistent with the README:** "framework exceptions now; DomainException + middleware mapping is my next step." Do not claim a DomainException that isn't in the code.
 - If you change anything after submitting, just say so in the review. Transparency scores.
-

6. Key questions — simple answers (from your code)

Why no MediatR? - At this size it adds indirection with no benefit — controllers are already thin. - If it grew (multiple steps, domain events, cross-cutting logging/validation), MediatR would earn its place.

What is DDD, and where did you apply it? - Designing code around the business, not the database. - Rich entities, factory Create(), guard clauses, private setters, Attendee inside the CalendarEvent aggregate.

What is an aggregate root? Why is Attendee inside it? - The root owns a group of objects and controls access. - Attendee has no meaning without an event; going through AddAttendee() enforces the rules (no duplicates, not on a cancelled event).

Explain RowVersion / optimistic concurrency. - Optimistic = assume clashes are rare, no locks, detect at save time. - SQLite has no native token, so it's a Guid I

regenerate on each write, marked `IsConcurrencyToken()`. - Client sends back its `RowVersion`; mismatch → 409. (Be ready for Gaps 1 and 2.)

Why SQLite? Why private constructors? Why records for DTOs? - SQLite: no server, self-creating DB; production would swap the provider only. - Private constructors + factory: all creation goes through validation; entity protects its own rules. - Records: value equality and init-only — data, not behaviour; annotations still validate at the boundary.

What are DI service lifetimes? (likely question) - Transient: a new instance every time it's requested. - **Scoped:** one instance per HTTP request, shared within that request. - **Singleton:** one instance for the whole app, shared by everyone. - In my code: the repository and notification service are `AddScoped`; the `DbContext` is scoped by default (`AddDbContext`). - **Why scoped for DbContext:** it represents one unit of work per request and isn't thread-safe, so all repository calls in a request share the same context and transaction. - **The trap:** never inject a scoped service (like `DbContext`) into a singleton — the singleton holds one forever and shares it across requests and threads, which corrupts state.

What is middleware? (and why exceptions belong there) - Middleware is a chain of components every request passes through before it reaches the controller — and the response passes back out through them. - Each one can inspect, change, or stop the request. Examples: HTTPS redirect, authentication, routing. **Order matters** (e.g. authentication before authorisation). - For exceptions, the clean approach is **one exception-handling middleware** (or `ExceptionHandler` in .NET 8) at the top of the pipeline. It catches errors from anywhere below and maps them — domain rule → 400, concurrency conflict → 409, not found → 404 — instead of every controller writing `try/catch`. - **Honest line:** "I didn't add exception-handling middleware in the time, so domain errors return 500 right now. Adding it, with `DomainException` types, is the clean fix and it's in my README."

Why Serilog instead of the default .NET logger? (likely question) - The built-in `ILogger` is fine but writes mostly plain text. - **Serilog does structured logging** — each log is data with named properties, so you can search and filter by them (e.g. all logs for one `EventId` or one request). - It supports **sinks** (console, file, Seq, Elasticsearch) and **enrichers** (auto-add correlation IDs, request info, machine name). - **Why it matters here:** a healthcare/regulated API needs traceable, queryable logs and an audit trail; structured logs plus correlation IDs make debugging far easier. - **Honest line:** "I used the default logger for the stub; adding Serilog for structured logging and proper sinks is on my README list."

EF Core quick answers: - *Migration:* tracked snapshot of schema changes; applied on startup here. - *AsNoTracking:* skips change-tracking for read-only queries; faster. I'd add it on GETs. - *IsConcurrencyToken:* EF adds `WHERE RowVersion = @original` to the `UPDATE`; no match → `DbUpdateConcurrencyException`. - *Eager vs lazy:* I use eager (`.Include`) to avoid N+1.

How would you scale / secure this? - SQL Server or Postgres; `AsNoTracking` + pagination on reads; caching for hot reads. - Move notifications to a queue so they don't block the response. - Add JWT auth + `[Authorize]`, and exception-to-status mapping.

7. Testing — simple answers (from your code)

What did you test, and why? - I tested the **domain logic** with xUnit — the most important code, because it enforces the business rules. - The domain has **no dependencies** (no EF Core, no HTTP), so I can test it directly with **no mocks and no database** — fast and simple. - My tests cover: creating an event sets status to Scheduled, end-before-start throws, cancelling an already-cancelled event throws, adding a duplicate attendee email throws, and updating a cancelled event throws.

What is xUnit? - The test framework. [Fact] marks a single test; [Theory] runs the same test with different inputs.

What is Shouldly? - An assertion library that makes tests read like English and gives clear failure messages. - Example:

`calendarEvent.EventStatus.ShouldBe(EventStatus.Scheduled)` reads naturally. - On failure it tells you exactly what was expected vs what you got — clearer than `Assert.Equal`.

What is the AAA pattern? - **Arrange** — set up the objects and inputs. - **Act** — call the method being tested. - **Assert** — check the result. - Keeps every test readable and consistent (my tests follow this).

Unit vs integration tests? - **Unit:** one piece in isolation, no I/O, fast — what I have here on the domain. - **Integration:** components together, hitting a real database or HTTP endpoint, slower.

What would you add with more time? - Integration tests on the API using `WebApplicationFactory`. - A test for the **409 conflict** on a concurrent update. - Tests for the GET filter combinations. - If I test EF Core, I'd use a **real SQLite/Postgres via Testcontainers**, not the in-memory provider — in-memory doesn't enforce real constraints or concurrency, so it gives false confidence.

8. Which patterns did you use?

Used: Domain-Driven Design · Aggregate · Factory Method · Repository · Vertical Slice · Dependency Inversion/DI · Optimistic Concurrency (version token) · DTO · Guard clauses/encapsulation · EF Core `IEntityTypeConfiguration<T>`.

Deliberately NOT used (say this too): MediatR/mediator (controllers are thin) · full CQRS with a separate read store (not needed at this scale).

9. Does vertical slice help with merge conflicts? (Yes — explain it)

One huge controller: - Every operation lives in one file. - Two developers on two features edit the **same file**, often near the same usings/constructor/routes. - Git merges overlapping edits in one file → that's where conflicts come from.

Vertical slice (what you did): - Each operation is a **separate file in its own folder**. - Two developers on two features touch **different files** → clean merges, nothing overlaps. - Smaller blast radius: editing "update" can't break "create" — they share no code. - Smaller, easier pull requests.

Say: "I used vertical slices partly for team reasons. With one big controller, feature branches collide because everyone edits the same file. A controller per operation means two people on two features touch different files and merge cleanly."

Trade-off to admit: vertical slices allow some duplication (my DTO mapping repeats in Create and Update). You accept a little duplication for independence, and extract a shared mapper later if it hurts.

10. Architecture & repository — deliberate decisions

Why one project instead of many class libraries? - I know the usual layout — Domain, Application, Infrastructure as separate .csproj files so the compiler enforces boundaries. - I chose one project with folders **on purpose**: for a 4-hour task it's faster to build and easier to review. - The layers are still clean — Domain has no infrastructure dependencies. - **Trade-off**: the boundary is a convention, not compiler-enforced — nothing physically stops domain code referencing EF Core. If it grew, splitting into libraries is my first structural change (it's in my README).

Are namespaces enough, or are class libraries better? - Namespaces organise code and make it findable — but they **enforce nothing**. Any code in the same assembly can reference any other. - Class libraries add a **compile-time wall**: if Domain doesn't reference EF Core, it physically can't use it. - So: namespaces = organisation; class libraries = enforced boundaries. They do different jobs — it's not "one is better." - For this size, namespaces are enough and I keep layers clean by discipline. On a bigger/longer-lived system I'd split into libraries — or, lighter option, add **architecture tests** (e.g. NetArchTest) that fail the build if Domain references Infrastructure.

Dependency direction — current vs split: - **Current single project**: no project references at all — layers are folders, the boundary is convention. Domain stays clean because I kept it that way. - **If split**: Domain references nothing → Infrastructure references Domain → Api references both and wires them in Program.cs. Then the compiler enforces the direction.

Why EF Core configuration classes (IEntityTypeConfiguration<T>)? - I kept the EF mapping in separate config classes, loaded via ApplyConfigurationsFromAssembly. - Alternatives I avoided: cramming it all into OnModelCreating (gets huge), or data annotations on the entities. - I avoided data annotations **on purpose** — they pull persistence concerns into the domain entities; I want the domain to stay clean and not know about the database. - Config classes keep all mapping in Infrastructure, one class per entity, so the DbContext stays thin. - It's where I put RowVersion as IsConcurrencyToken(), the enum as a string via HasConversion<string>(), the max lengths, and the CalendarEvent→Attendees cascade.

Moving the repository interface to Domain — just the interface? - Yes — **interface to Domain, implementation stays in Infrastructure**. - Interface = the contract (a domain concern); implementation = how it's done with EF Core (infrastructure detail). - This is **dependency inversion**: Domain owns the contract and depends on nothing; Infrastructure depends on Domain. - I already do exactly this with INotificationService, so the repository should follow the same pattern.

Could you use a generic repository instead? (careful — different question) - Generic vs specific is about **shape**; interface placement is about **dependency direction**

— two different things, so one doesn't replace the other. - A generic IRepository<T> is usually **weaker in DDD**: it tends to expose IQueryable / GetAll, which lets callers bypass the aggregate — the opposite of what my design protects. - I deliberately kept a **specific** ICalendarEventRepository with intent-revealing methods (GetAllAsync(from, to, status), SearchAsync), one repository per aggregate root — the more DDD-aligned choice. - **Fair nuance**: generic repositories are fine for simple CRUD with no rich domain; for a rich aggregate, specific is better. Knowing *when* each applies is the point.

11. One-line mindset for the room

- Lead with strengths, then volunteer gaps with "here's what I'd do next."
- End each answer on a trade-off ("I chose X over Y because...").
- You documented decisions and reviewed your own code critically — that's the senior signal.